- Spécification OData & Fiori - ZPOCREATERAL
    - 1. Objet
    - 2. Objectif fonctionnel
    - 3. Acteurs
        - 3.1 Acteurs métier
        - 3.2 Acteurs techniques
    - 4. Architecture générale
    - 5. Service OData
        - 5.1 Service et container
        - 5.2 Implémentation backend
    - 6. Entity Sets exposés
        - 6.1 POHeaderSet
        - 6.2 POItemSet
        - 6.3 KPIDataSet
        - 6.4 ChartDataSet
        - 6.5 VendorHelperSet
        - 6.6 MaterialHelperSet
    - 7. Modèle de données fonctionnel
        - 7.1 POHeader
        - 7.2 POItem
        - 7.3 KPIData
        - 7.4 ChartData
    - 8. Backend ABAP
        - 8.1 Classe principale
        - 8.2 Helpers métiers
    - 9. Logique métier par flux
        - 9.1 Consultation des entêtes
        - 9.2 Consultation des postes
        - 9.3 Création via l'interface
        - 9.4 KPI
        - 9.5 Données analytiques
        - 9.6 Value helps
    - 10. Frontend Fiori / SAPUI5
        - 10.1 Structure applicative
        - 10.2 Écrans fonctionnels
        - 10.3 Description des écrans
    - 11. Étapes de mise en oeuvre

    - 11.1 Backend OData
    - 11.2 Frontend Fiori
    - 11.3 Vérifications après déploiement
- 12. Points d'attention techniques
    - 12.1 Contrôles frontend
    - 12.2 Contrôles backend
    - 12.3 Sécurité
- 13. Règles de gestion
    - 13.1 Consultation des PO
    - 13.2 Création via interface
    - 13.3 Consultation des postes
    - 13.4 Value helps
    - 13.5 Dashboard
- 14. Parcours de recette
    - 14.1 Démarrage applicatif
    - 14.2 Consultation
    - 14.3 Création via interface
    - 14.4 Flux spécifiques
- 15. Conclusion

Spécification OData & Fiori - ZPOCREATERAL

1. Objet

Cette spécification décrit l'application Fiori `ZPOCREATERAL` et son service OData associé. Elle couvre :

- le périmètre fonctionnel côté SAPUI5 / Fiori
- les entity sets OData exposés
- les rôles métier couverts
- l'architecture technique backend / frontend
- les principales règles de gestion
- les points d'attention de mise en oeuvre

Le service OData central du projet est `ZPOCREATERAL`.

2. Objectif fonctionnel

L'application a pour objectif de permettre :

- la consultation des commandes d'achat
- la consultation des postes de commande
- la création de commandes d'achat via l'interface
- la consultation d'indicateurs achats
- la recherche de fournisseurs
- la recherche d'articles

L'application est pensée comme une application Fiori multi-écrans avec un backend Gateway unique.

3. Acteurs

3.1 Acteurs métier

- `Acheteur` : consulte et crée les commandes d'achat
- `Responsable achats` : suit les indicateurs et l'activité fournisseurs

3.2 Acteurs techniques

- service Gateway `ZPOCREATERAL`
- classe backend `ZCL_ZPOCREATE_RAL_DPC_EXT`
- sources standard SAP : `EKKO`, `EKPO`, `LFA1`, `MARA`, `MAKT`
- frontend `SAPUI5 / Fiori`

4. Architecture générale

L'architecture repose sur deux couches :

- un frontend `SAPUI5 / Fiori` avec navigation multi-écrans
- un backend `SAP Gateway OData` exposant le service `ZPOCREATERAL`

Le backend centralise :

- la lecture des données d'achat
- la création de commandes d'achat via l'interface
- les value helps fournisseurs et articles
- les KPI et jeux de données analytiques

5. Service OData

5.1 Service et container

- service : `ZPOCREATERAL`
- container : `ZPOCREATERALEntities`

5.2 Implémentation backend

- classe MPC : définit le modèle OData
- classe DPC_EXT : implémente la logique de chaque endpoint

Classe principale :

- `ZCL_ZPOCREATE_RAL_DPC_EXT`

6. Entity Sets exposés

6.1 `POHeaderSet`

Rôle :

- lecture de la liste des commandes d'achat
- lecture du détail d'une commande
- création d'une commande d'achat via l'interface

Opérations :

- `GET`

- `GET(key)`
- `POST`

Filtres principaux :

- `Vendor`
- `CompCode`
- `DocDate`

Remarque :

- le `POST` sur `POHeaderSet` est utilisé comme flux de création depuis l'interface
- la création peut être portée par un payload header avec items

6.2 `POItemSet`

Rôle :

- lecture des postes de commande

Opérations :

- `GET`
- `GET(key)`

Recherche :

- par `PoNumber`
- ou par clé composée `PoNumber` / `PoItem`

6.3 `KPIDataSet`

Rôle :

- alimentation des KPI du dashboard

Opérations :

- `GET`
- `GET(key)`

Indicateurs principaux :

- `TOTALPO`
- `POMONTH`
- `AMOUNTMONTH`
- `TOTALAMOUNT`

6.4 `ChartDataSet`

Rôle :

- alimentation des graphiques analytiques

Opération :

- `GET`

Types de graphiques couverts :

- `POTREND`
- `VENDORPIE`

6.5 `VendorHelperSet`

Rôle :

- aide à la saisie fournisseur

Opération :

- `GET`

Source :

- `LFA1`

6.6 `MaterialHelperSet`

Rôle :

- aide à la saisie article

Opération :

- `GET`

Sources :

- `MARA`
- `MAKT`

7. Modèle de données fonctionnel

7.1 `POHeader`

Champs principaux :

- `PoNumber`
- `DocType`
- `Vendor`
- `PurchOrg`
- `PurGroup`
- `CompCode`
- `Currency`
- `DocDate`
- `PmntTrms`
- `Status`
- `StatusText`
- `CreatedBy`
- `CreatedAt`
- `TotalAmount`
- `ItemCount`

Navigation :

- `ToItems` vers `POItem`

7.2 `POItem`

Champs principaux :

- `PoNumber`

- `PoItem`
- `Material`
- `MaterialDesc`
- `Plant`
- `Quantity`
- `PoUnit`
- `DelivDate`
- `NetPrice`
- `PriceUnit`
- `Currency`
- `ItemCat`
- `AcctAssCat`
- `StorLoc`
- `Status`

7.3 `KPIData`

Champs principaux :

- `KpiId`
- `Title`
- `Value`
- `ValueFormatted`
- `Unit`
- `Trend`
- `TrendValue`
- `Color`
- `Icon`
- `Period`

7.4 `ChartData`

Champs principaux :

- `ChartType`
- `DimKey`
- `DimLabel`

- `Measure1`
- `Measure2`
- `Measure3`
- `Label1`
- `Label2`
- `Label3`
- `Color1`

8. Backend ABAP

8.1 Classe principale

La classe `ZCL_ZPOCREATE_RAL_DPC_EXT` hérite de la DPC générée et concentre la logique métier OData.

Méthodes Gateway mentionnées dans la spécification d'origine :

- `CHARTDATASETGETENTITYSET`
- `KPIDATASETGETENTITY`
- `KPIDATASETGETENTITYSET`
- `MATERIALHELPERSETGETENTITYSET`
- `POHEADERSETCREATEENTITY`
- `POHEADERSETGETENTITY`
- `POHEADERSETGETENTITYSET`
- `POITEMSETGETENTITY`
- `POITEMSETGETENTITYSET`
- `VENDORHELPERSETGETENTITYSET`

8.2 Helpers métiers

Helpers privés mentionnés :

- `CREATEPOFROMDATA`
- `READPOHEADER`
- `MAPEKKOTOPOHEADER`

Leur rôle est de séparer :

- le contrat OData
- la logique métier
- le mapping des données SAP

9. Logique métier par flux

9.1 Consultation des entêtes

`POHEADERSETGETENTITYSET` :

- lit les filtres Gateway
- interprète les bornes de date
- délègue la lecture à `READPOHEADER`

`READPOHEADER` :

- lit `EKKO`
- filtre les commandes d'achat actives
- applique les critères optionnels
- trie les résultats
- limite la volumétrie

`MAPEKKOTOPOHEADER` :

- enrichit chaque entête
- calcule le montant total
- calcule le nombre de postes
- positionne le statut métier

9.2 Consultation des postes

`POITEMSETGETENTITYSET` :

- exige un `PoNumber`
- évite les lectures massives sans filtre
- lit les postes via `EKPO`
- enrichit la description article via `MAKT`

`POITEMSETGETENTITY` :

- lit un poste unitaire par clé composée

9.3 Création via l'interface

`POHEADERSETCREATEENTITY` :

- lit les données saisies dans l'écran de création
- récupère les informations header
- récupère les informations item nécessaires à la création
- délègue la création à `CREATEPOFROMDATA`

`CREATEPOFROMDATA` :

- prépare les structures `BAPIMEPOHEADER` et `BAPIMEPOHEADERX`
- prépare les structures `BAPIMEPOITEM` et `BAPIMEPOITEMX`
- appelle `BAPI_PO_CREATE1`
- retourne le numéro de PO créé et le statut de traitement

Le flux de création décrit ici est un flux **interface utilisateur**, distinct d'un import CSV.

9.4 KPI

`KPIDATASETGETENTITYSET` :

- calcule les bornes du mois courant et précédent
- produit les KPI de volume et de montant
- calcule la tendance
- alimente couleur et icône

`KPIDATASETGETENTITY` :

- lit un KPI unitaire par identifiant

9.5 Données analytiques

`CHARTDATASETGETENTITYSET` :

- supporte `POTREND`
- supporte `VENDORPIE`

`POTREND` :

- agrégation mensuelle glissante
- volume et montant par période

`VENDORPIE` :

- top fournisseurs par volume de commandes

9.6 Value helps

`VENDORHELPERSETGETENTITYSET` :

- recherche par code ou nom
- limite le nombre de résultats
- exclut les fournisseurs bloqués

`MATERIALHELPERSETGETENTITYSET` :

- recherche par code ou description
- exclut les articles supprimés ou bloqués
- fallback anglais si la langue de session est absente

10. Frontend Fiori / SAPUI5

10.1 Structure applicative

L'application est conçue comme un composant routé multi-cibles.

Éléments structurants attendus :

- `Component.js`
- `manifest.json`
- vue racine `App.view.xml`
- un conteneur de navigation de type `sap.m.App` ou `sap.m.NavContainer`
- un `controlId` cohérent avec le routeur, généralement `app`

10.2 Écrans fonctionnels

Écrans cibles mentionnés :

- `Dashboard`
- `CreatePo`
- `PoList`
- `Vendors`
- `Materials`

10.3 Description des écrans

`Dashboard`

- affiche les KPI
- affiche la tendance mensuelle
- affiche le top fournisseurs

`Création PO`

- formulaire de création d'une commande d'achat
- sélection fournisseur via `VendorHelperSet`
- sélection article via `MaterialHelperSet`
- saisie des informations de quantité, organisation et société
- soumission vers `POHeaderSet`

`Liste des PO`

- filtres par fournisseur, société et date
- liste des entêtes via `POHeaderSet`
- détail des postes via `POItemSet`

`Fournisseurs`

- recherche fournisseur via `VendorHelperSet`

`Articles`

- recherche article via `MaterialHelperSet`

11. Étapes de mise en oeuvre

11.1 Backend OData

1. Créer ou vérifier le projet Gateway `ZPOCREATERAL`
2. Définir les entity types et entity sets dans la MPC
3. Implémenter la logique dans `ZCL_ZPOCREATE_RAL_DPC_EXT`
4. Générer et enregistrer le service
5. Activer et publier le service OData

11.2 Frontend Fiori

1. Créer le composant SAPUI5
2. Définir le `manifest.json`
3. Déclarer les routes et targets
4. Créer la vue racine avec l'id attendu par le routeur
5. Connecter le modèle OData au service `ZPOCREATERAL`
6. Créer les vues `Dashboard`, `CreatePo`, `PoList`, `Vendors`, `Materials`

11.3 Vérifications après déploiement

Vérifier :

- que le service OData répond
- que les entity sets sont exposés
- que la navigation frontend démarre sans erreur
- que le fichier `model/menu.json` est présent si attendu par le shell
- que les routes du manifest pointent vers un `controlId` réellement existant

12. Points d'attention techniques

12.1 Contrôles frontend

Erreurs connues mentionnées dans la spécification source :

- `Control with ID app could not be found`
- `Cannot GET model/menu.json`
- fallback locale incomplet
- fichiers Flex absents (`flexibility-bundle.json`, `changes-bundle.json`)

12.2 Contrôles backend

Points d'attention :

- la création via l'interface doit rester cohérente avec le contrat OData exposé par `POHeaderSet`
- le mapping entre données d'écran et structures BAPI doit être maîtrisé
- la gestion des messages de retour doit être homogène entre succès et échec

12.3 Sécurité

La spécification source signale qu'aucune stratégie complète `AUTHORITY-CHECK` / `PFCG` n'est documentée à ce stade.

13. Règles de gestion

13.1 Consultation des PO

- lecture des entêtes depuis `EKKO`
- enrichissement depuis `EKPO`
- statut fonctionnel retourné pour les commandes actives

13.2 Création via interface

- la création est initiée depuis l'écran `CreatePo`
- l'utilisateur renseigne le header et les données article nécessaires
- le backend transforme la saisie interface en structures BAPI
- la création retourne un numéro de commande et un statut fonctionnel

13.3 Consultation des postes

- lecture détaillée des postes depuis `EKPO`
- enrichissement description article via `MAKT`

13.4 Value helps

- recherche fournisseur via `LFA1`
- recherche article via `MARA/MAKT`

13.5 Dashboard

- KPI de volume
- KPI de montant
- tendance mensuelle
- top fournisseurs

14. Parcours de recette

14.1 Démarrage applicatif

Vérifier :

- chargement du shell
- absence d'erreur de routing
- chargement du menu
- affichage du dashboard

14.2 Consultation

Vérifier :

- affichage des KPI
- affichage des graphiques
- affichage de la liste des commandes
- affichage des postes d'une commande
- recherche fournisseur

- recherche article

14.3 Création via interface

Vérifier :

- chargement de l'écran `CreatePo`
- alimentation des aides à la saisie fournisseur et article
- soumission du formulaire
- retour du numéro de commande ou du message d'erreur métier

14.4 Flux spécifiques

Vérifier selon le périmètre projet :

- cohérence du dashboard
- cohérence des value helps
- navigation entre liste et détail

15. Conclusion

`ZPOCREATERAL` est conçu comme un service OData unique supportant une application Fiori achats multi-écrans. Le backend Gateway centralise la logique d'accès aux données, la création de PO via l'interface, les value helps et les indicateurs analytiques. Le frontend SAPUI5 s'appuie sur ce service pour fournir un parcours utilisateur unifié autour de la consultation, de la création et du suivi des commandes d'achat.